



NTU Academy for Professional
and Continuing Education

(SCTP) Advanced
Professional Certificate

Data Science and AI





NANYANG
TECHNOLOGICAL
UNIVERSITY
SINGAPORE

3.8 Computer Vision

Module Overview

3.1 Probability and Statistics for Machine Learning

3.2 Introduction to Machine Learning

3.3 Supervised Learning

3.4 Supervised Learning - Advanced

3.5 Unsupervised Learning

3.6 Time Series Data & Forecasting

3.7 Neural Network & Deep Learning

3.8 Computer Vision (CV)

3.9 Natural Language Processing (NLP)

3.10 NLP - Advanced

Lesson Objectives

- Apply classical image processing techniques to images
- Create and train a CNN for image classification
- Utilize pre-trained models for image classification

Lesson Plan

- Introduction to CV and Image Processing
- Training CNNs
- Transfer Learning and Pre-Trained Models

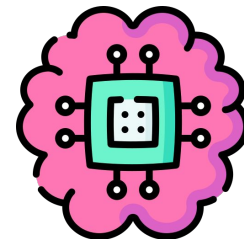
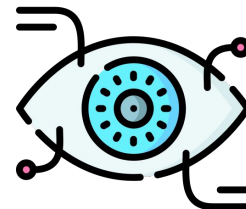


Computer Vision

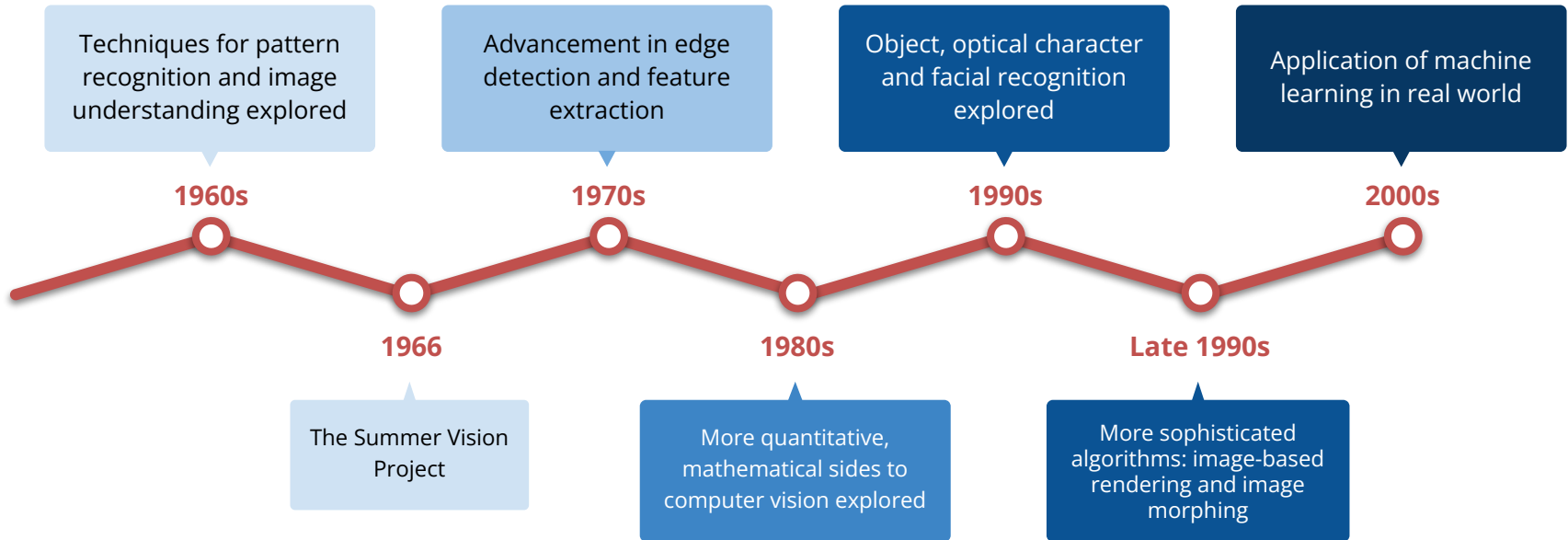
Computer vision is a field of artificial intelligence (AI) that enables computers to interpret and understand visual information from the world—like images or videos—just like humans do.

In more practical terms, it involves teaching computers to:

- **See:** Capture images or video using cameras or sensors.
- **Process:** Analyze the content of those images (e.g., detecting edges, patterns, or shapes).
- **Understand:** Recognize objects, classify scenes, detect movements, or even track people or things.
- **Act:** Make decisions based on the visual input (e.g., a self-driving car stopping for a pedestrian).



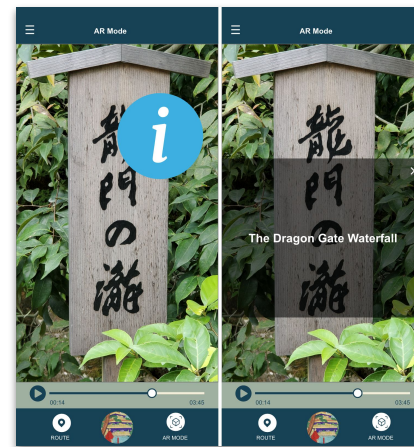
History of Computer Vision



CV Applications: Image-based Translations

Translates text seen through a camera in real time—like a street sign, menu, or document.

- **Text detection:** CV locates the words in the image (using OCR—optical character recognition).
- **Text recognition:** The characters are recognized and turned into digital text.
- **Language translation:** NLP (natural language processing) translates the text.
- **Rendering:** The translated words are overlaid back onto the original image, often matching font and background.



CV Applications: Object Detection

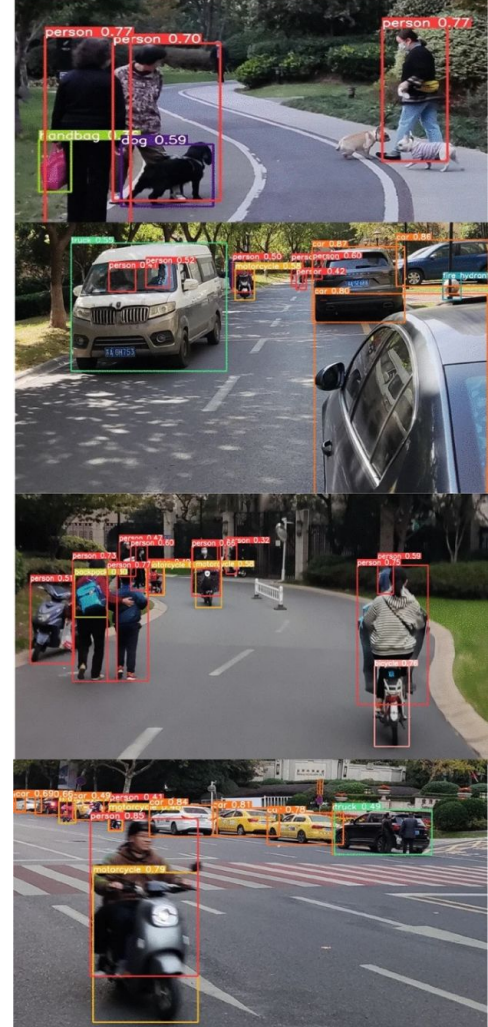
Detects and identifies multiple objects in an image or video, and shows where they are with bounding boxes.

- Combines classification (*what is it?*) with localization (*where is it?*).
- Uses models like: **YOLO** (You Only Look Once) – real-time, super fast
- Outputs include:
 - Object class (e.g., "cat", "car", "person")
 - Confidence score (e.g., 94% sure it's a dog)
 - Coordinates for the object's location

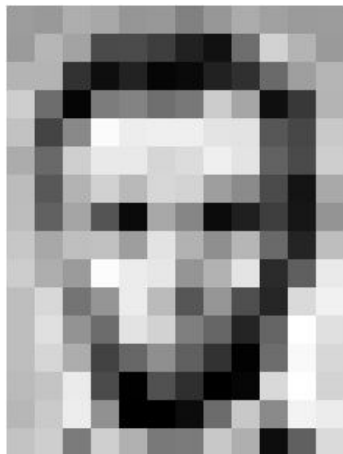
Used in:

- **Autonomous vehicles:** Detect pedestrians, other cars, traffic signs
- **Retail:** Track products on shelves
- **Surveillance:** Spot intruders or unusual activity
- **Agriculture:** Count fruits or detect crop diseases

<https://www.nature.com/articles/s41598-024-51868-0>



What you see VS what the computer sees



157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	83	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	228	227	87	71	201	
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218



GRAY (137)



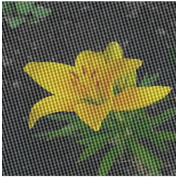
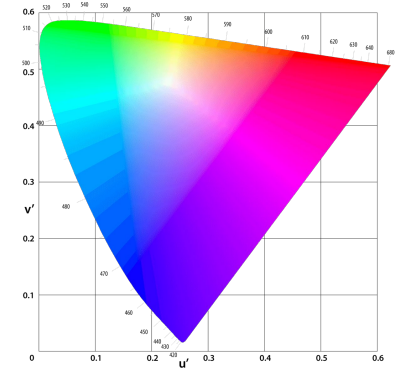
R (141)
G (138)
B (133)

https://openframeworks.cc/ofBook/chapters/image_processing_computer_vision.html

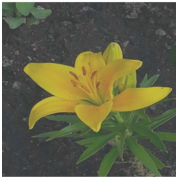
Image Processing Techniques

There are several basic techniques used in image processing that help in achieving various goals:

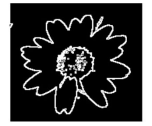
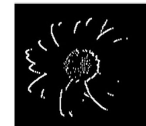
1. **Color Space Conversions:** This includes changing the brightness, contrast, or saturation of an image. Adjusting colors can make images more visually appealing and highlight important features.
2. **Filtering:** This technique involves removing noise or unwanted elements from an image. For example, a filter can smooth out grainy images or sharpen blurry ones.
3. **Edge Detection:** This method is used to identify the boundaries within an image. By detecting edges, you can find the outlines of objects within a photo, which is crucial for recognizing shapes and patterns.



(a) Corrupted yellow lily

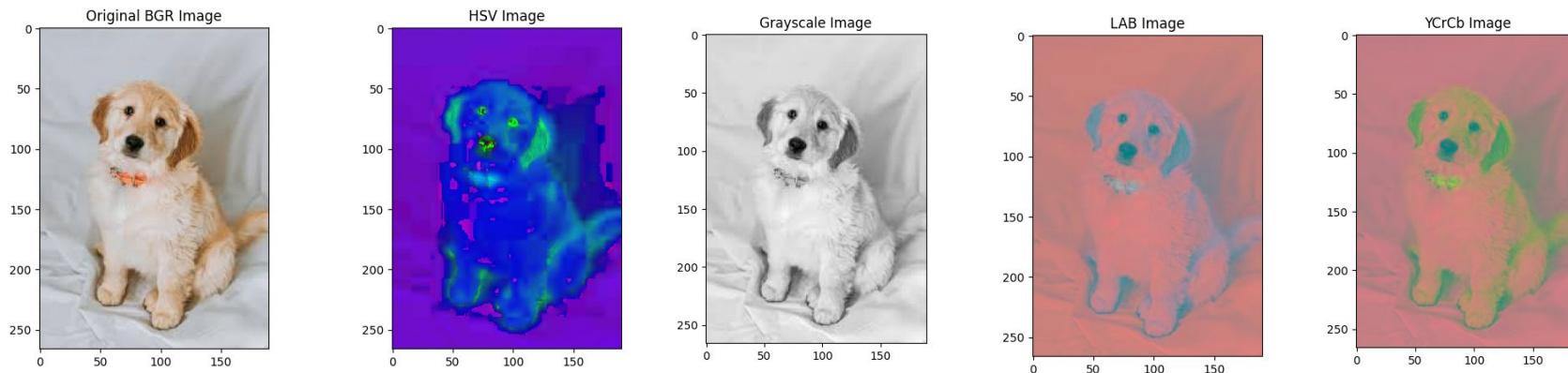


(b) De-noised yellow lily



Color Space Conversions

A color space is a specific way of organizing and representing colors. Different color spaces make different tasks easier—for example, detecting skin tones, isolating bright objects, or enhancing contrast.



<https://blog.roboflow.com/opencv-color-spaces/>

Color Space Conversions

A color space is a specific way of organizing and representing colors. Different color spaces make different tasks easier—for example, detecting skin tones, isolating bright objects, or enhancing contrast.

Task	Preferred Color Space	Why It's Preferred
Skin detection	HSV / YCrCb	Separates color from brightness, making skin tones easier to isolate
Object color tracking	HSV	Hue is stable under lighting changes—great for tracking consistent colors
Edge/shape detection	Grayscale	Simplifies the image to intensity only, making edge detection faster and more effective
Color correction	LAB	Mimics human vision and isolates lightness, enabling precise color adjustments
Compression (video)	YCrCb	Efficiently separates luminance from chrominance—better compression with less loss

Image Filtering and Convolution

Filtering is the process of modifying an image to enhance or suppress certain features. It can be used to sharpen edges, blur noise, or highlight patterns.

You apply a filter (also called a *kernel* or *mask*) to an image, usually using convolution.

Filter Type	Purpose	Example Kernel (3×3)
Blur / Smoothing	Reduce noise, soften image	All 1s: $\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} / 9$
Sharpen	Enhance edges, highlight details	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$
Edge Detection	Find object boundaries	Sobel, Prewitt, or Laplacian filters
Emboss	Create a 3D-like shaded effect	$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$
Gaussian Blur	Smooth while preserving edge structure	Weighted Gaussian kernel

Image Filtering and Convolution

Name	Matrix	clamp	wrap	mirror	omit	0.0	1.0
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$						
Box Blur	$\begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}$						
Gaussian Blur	$\begin{bmatrix} 1/16 & 1/8 & 1/16 \\ 1/8 & 1/4 & 1/8 \\ 1/16 & 1/8 & 1/16 \end{bmatrix}$						
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$						
Laplacian	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$						

<https://dev.to/ndesmic/playing-with-image-convolution-filters-5a2e>

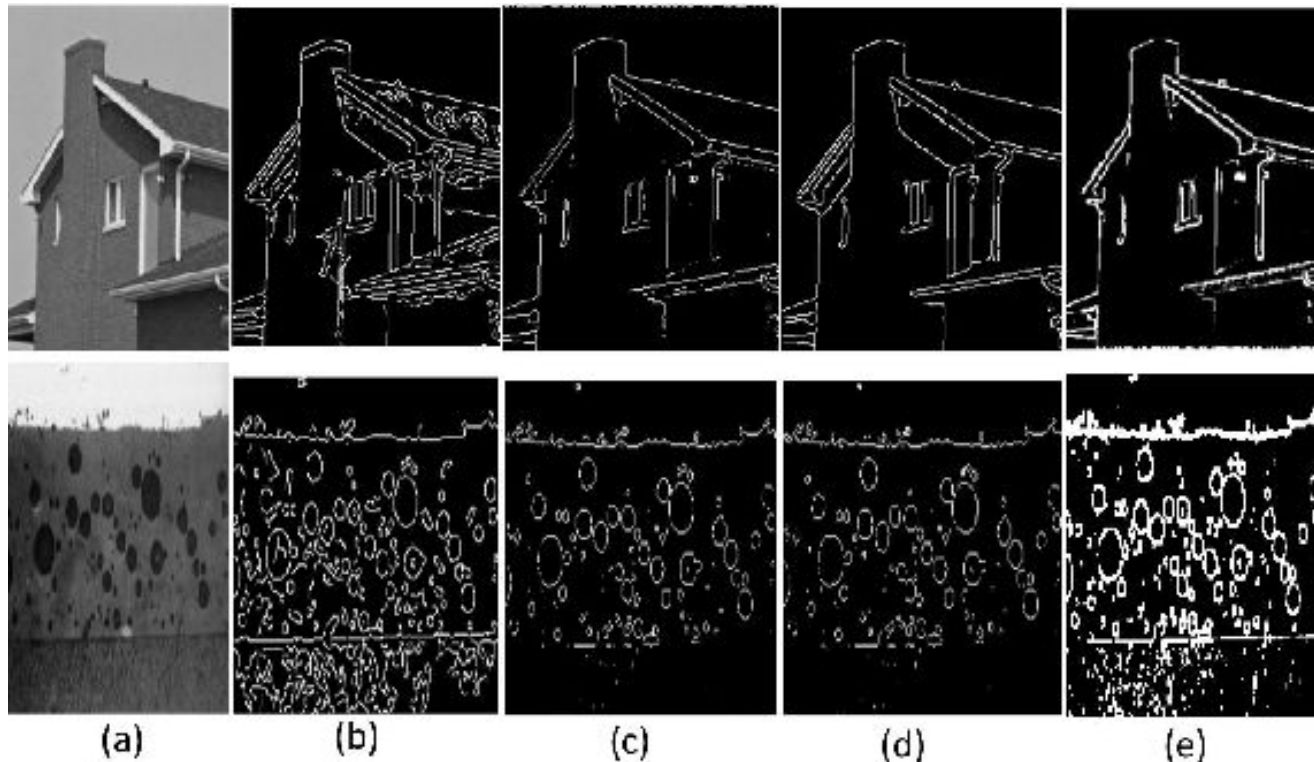
Edge Detection

Edge detection is the process of identifying the boundaries or edges of objects within an image. Edges represent the points where significant changes in pixel intensity occur, and they help in distinguishing different regions of an image. Sobel and Canny are two popular methods for edge detection.

Edge Detection Method	Purpose	Use Case
Sobel Operator	Detects edges using gradients	General purpose, edge detection in images
Canny Edge Detection	High-precision edge detection	Complex images, medical imaging, object tracking
Prewitt Operator	Detects edges with simpler kernels	Faster edge detection in simple images
Laplacian of Gaussian	Detects sharp intensity changes	Edge detection in noisy or complex images

Edge Detection

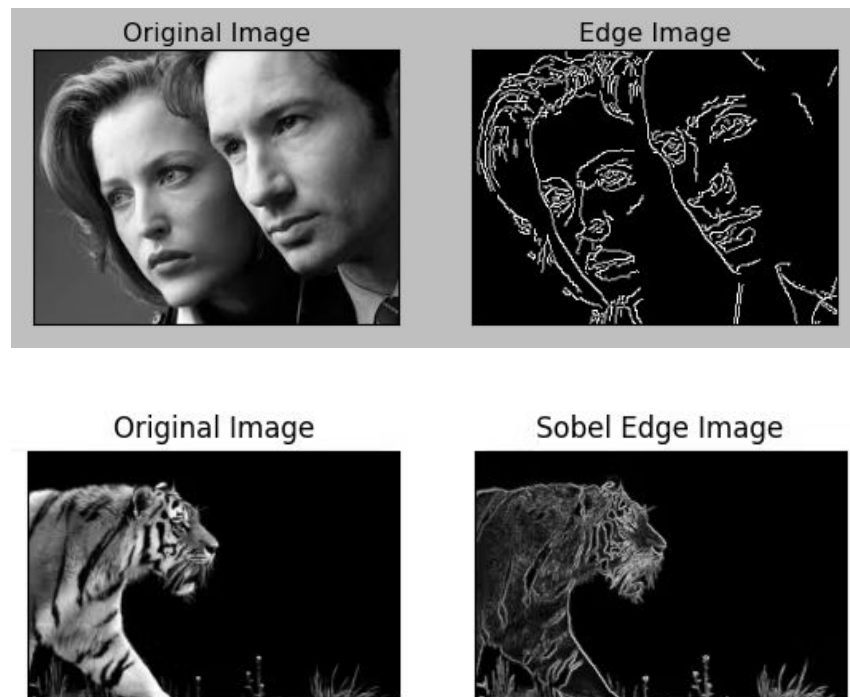
- (a) Test image
- (b) Canny's method
- (c) Sobel' method
- (d) An's method
- (e) Bat Algorithm.



https://www.researchgate.net/figure/Edge-detection-by-different-methods-a-Test-image-b-Cannys-method-c-Sobel-method_fig3_316562171

Why Is Edge Detection Important?

1. **Object Detection:** Identifying boundaries of objects in images or videos.
2. **Image Segmentation:** Splitting an image into meaningful parts based on edges.
3. **Feature Extraction:** Extracting key features like shapes and lines.
4. **Simplification:** Reducing the complexity of the image by focusing on key features.



Code-along



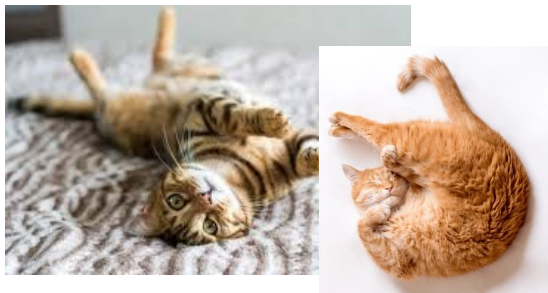
Jupyter Notebook

Part 1: Image Processing

How does AI classify an image?



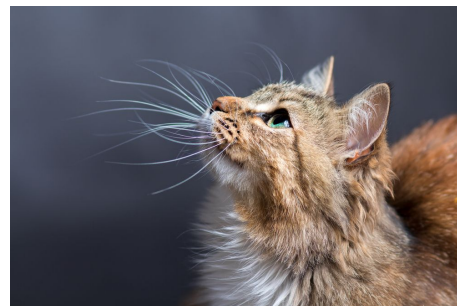
Potential Problems?



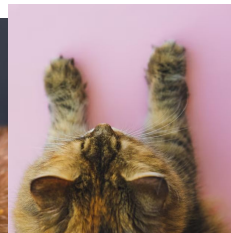
Distortion



Occlusion



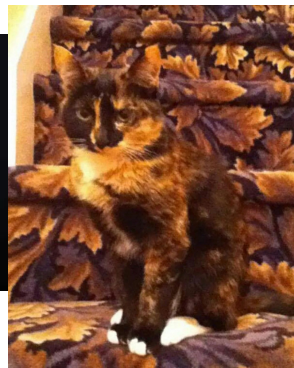
Viewpoint variation



Illumination



Scale

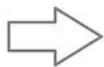


Background Noise

Why CNNs for Computer Vision?

CNN
Or
Feedforward
Network?

1	1	0
4	2	1
0	2	1



1
1
0
4
2
1
0
2
1



Considerations



Compute

Generalization

Correlations

What is “Convolution”

Input image



Convolution
Kernel

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Feature map

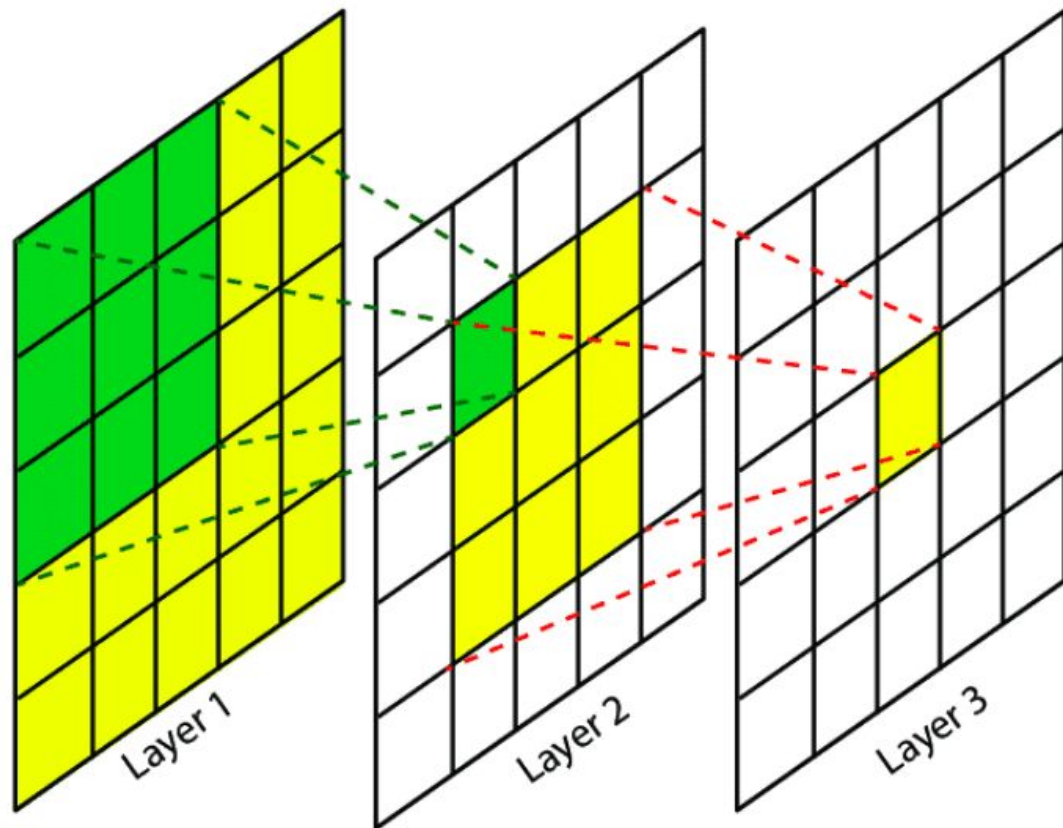


<https://developer.nvidia.com/discover/convolution>

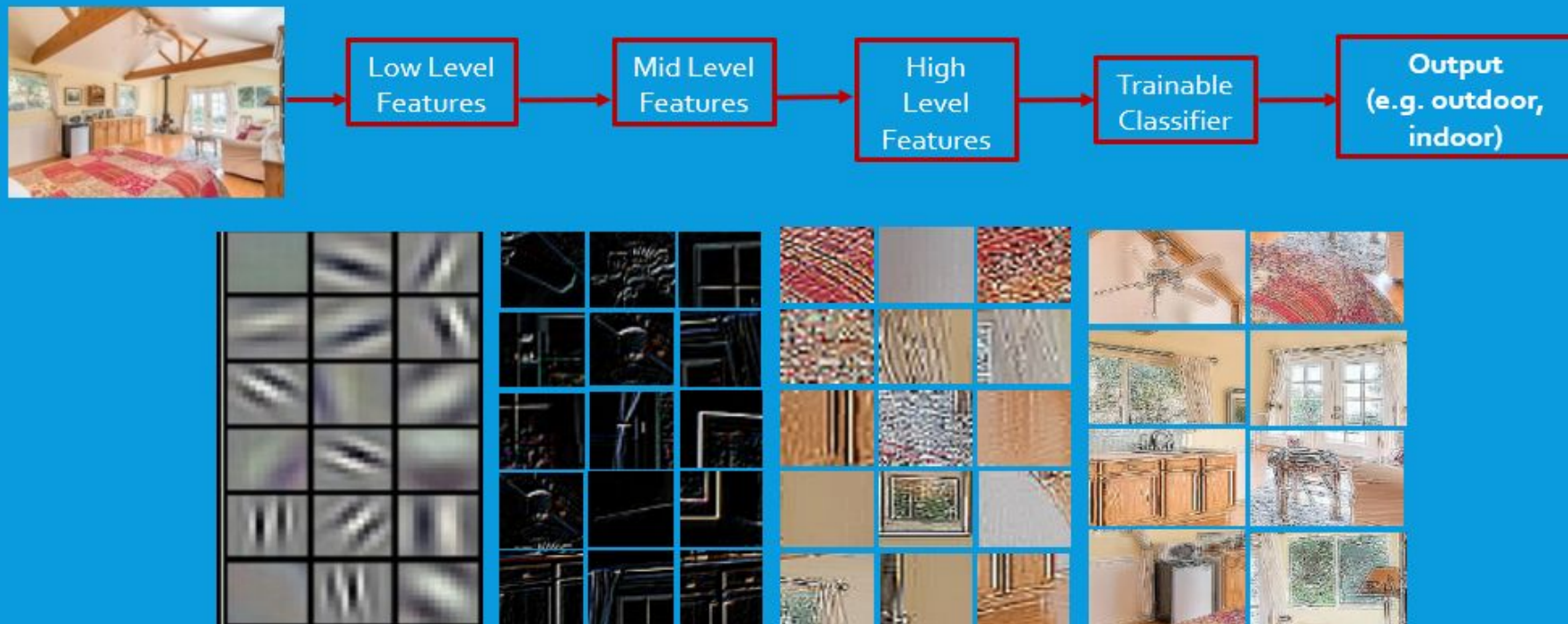
Examples of Convolution Kernels / Filters >>

Operation	Filter	Convolved Image
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

<https://ujjwalkarn.me/2016/08/11/intuitive-explanation-convnets/>

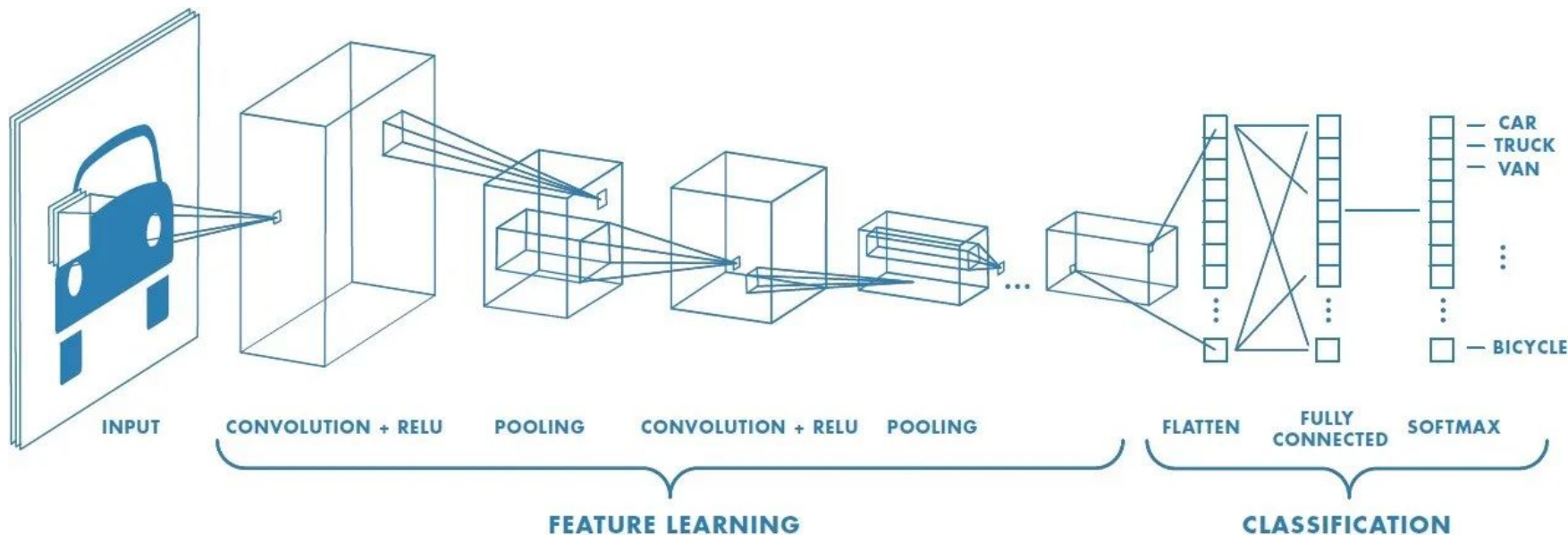


How does AI classify an image?



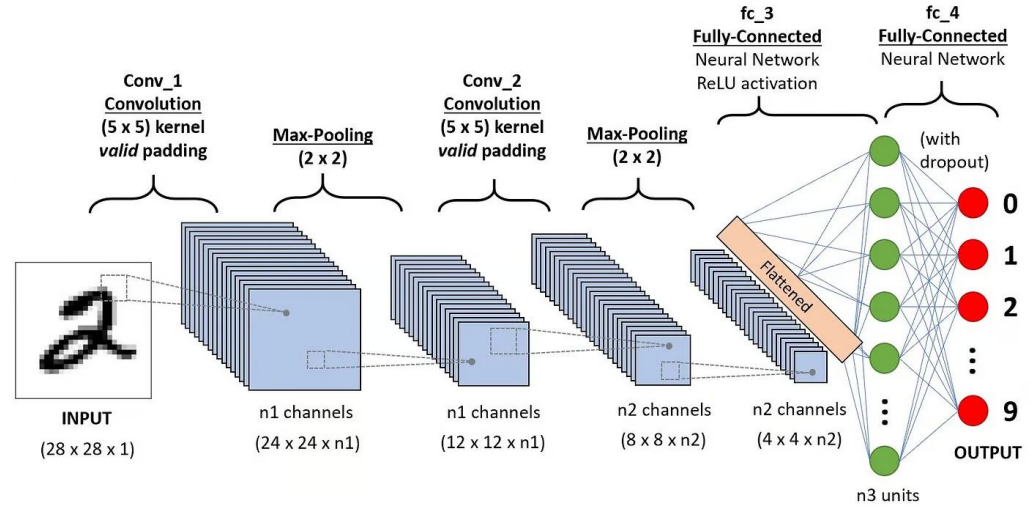
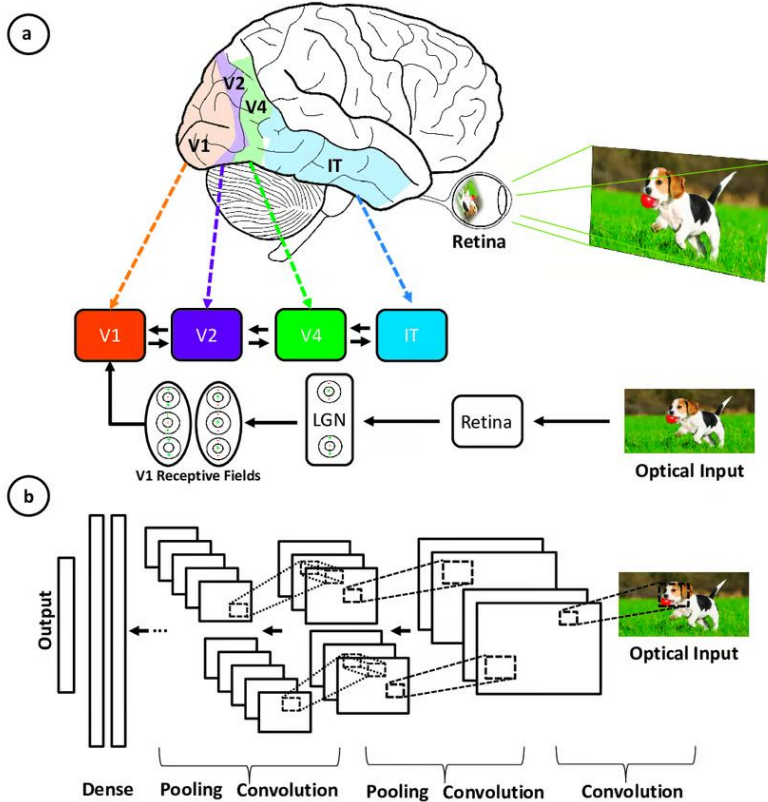
Convolutional Neural Networks - CNNs

CNNs are a class of deep neural networks, specifically designed for Computer Vision tasks. They have been highly successful in various computer vision tasks and are the backbone of many state-of-the-art computer vision applications.

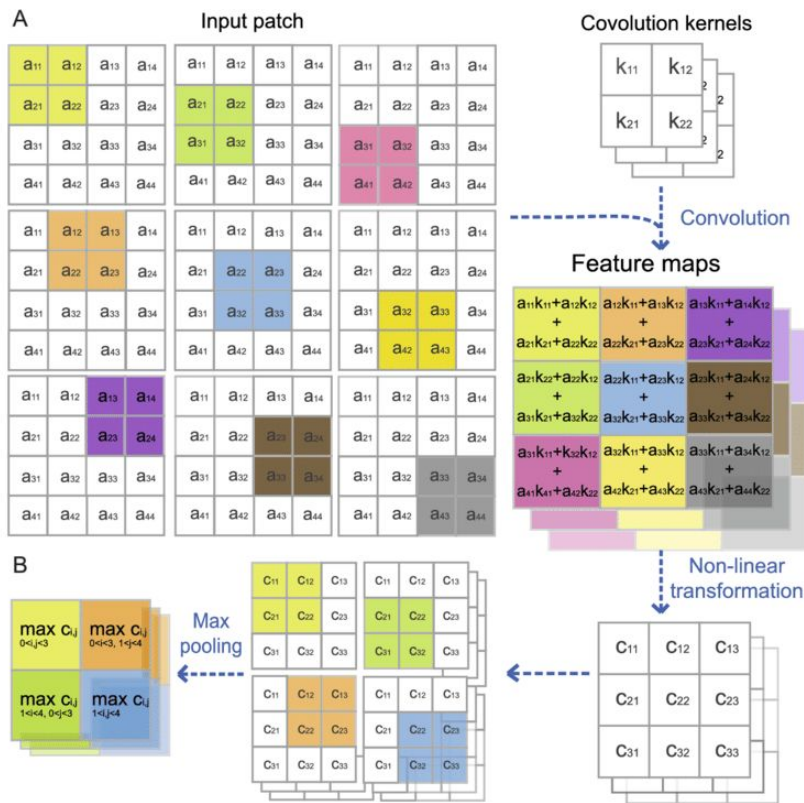


<https://medium.com/data-science/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53>

Convolutional Neural Networks - CNNs



Convolutional Layers



The convolutional layer is the first layer of a convolutional network.

- It takes the raw input image (pixels) and applies filters (tiny matrices) to detect simple patterns like edges, lines, or color gradients.
- Think of it like a visual scanner that's looking for very basic shapes.
- Each deeper layer learns more complex patterns by combining what previous layers saw.

Kernel Limitations

1x1 Kernel

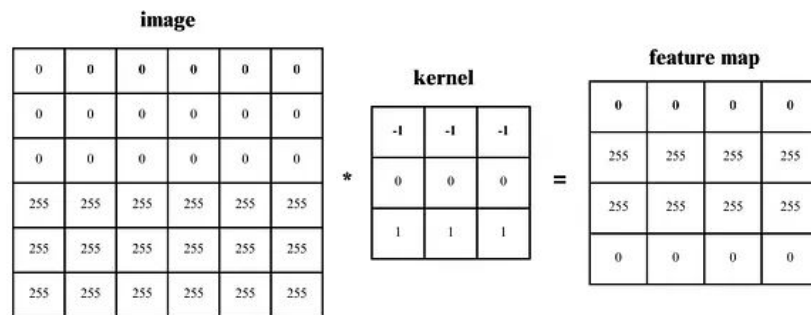
Effectively just performs a linear combination of the input features at each spatial location.

It doesn't capture spatial relationships or local patterns because it only looks at a single pixel at a time.

2x2 Kernel

Captures a small, localized region of the input, but it's still too small to effectively capture complex patterns or high-level features.

It also introduces a lot of parameter redundancy and inefficiency for capturing larger spatial patterns.



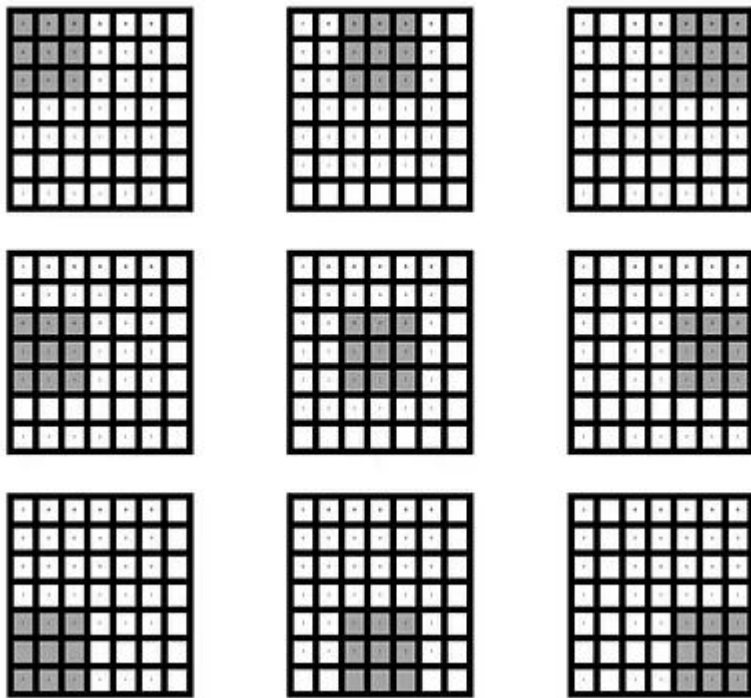
4x4 Kernel

Captures a larger region of the image compared to 1x1 or 2x2 kernels.

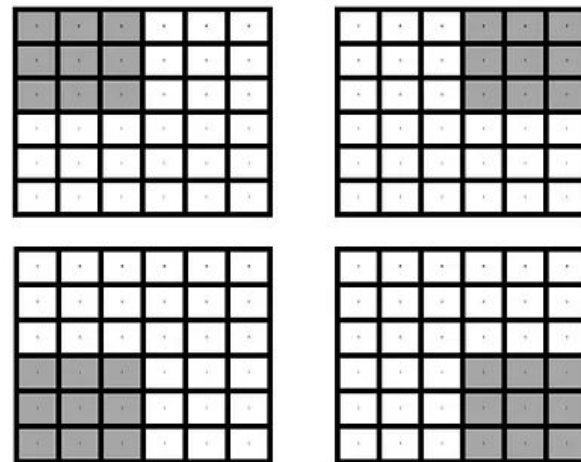
However, 2x2 and 4x4 are generally not preferred because odd-sized filters symmetrically divide the previous layer pixels around the output pixel.

If this symmetry is not present, there will be distortions across the layers which happens when using an even sized kernels

Stride



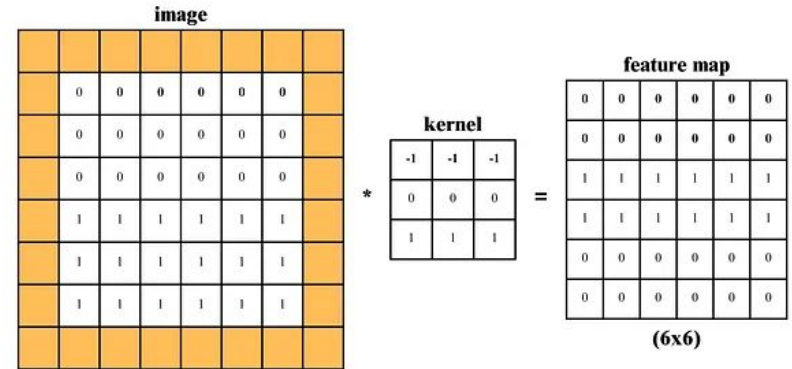
Stride = 2



Stride = 3

Padding

- Preserving Spatial Dimensions:** Padding ensures that the spatial dimensions (height and width) of the input feature maps are preserved after applying convolutions. Without padding, the spatial dimensions would shrink after each convolutional operation. This can lead to a loss of information, especially at the edges of the input.
- Edge Information:** Padding helps preserve information at the borders or edges of the input image. When the convolutional kernel is applied near the boundaries, some parts of the kernel might not have access to data outside the image, so padding fills those gaps.
- Control over Output Size:** Padding allows control over the size of the output feature maps. Without padding, the output size decreases with each convolution, but by adding padding, you can ensure the output size is either the same as the input or controlled to a specific dimension.



Pooling Layers

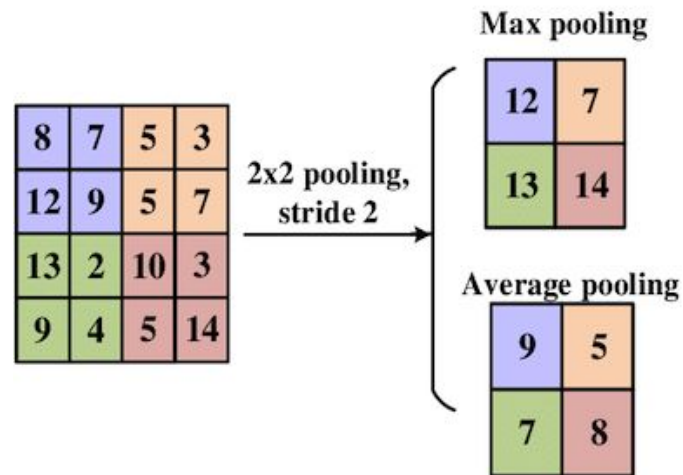
Pooling layers play a crucial role in convolutional neural networks by reducing the spatial size of the representation and the number of parameters, which helps control overfitting. They work by summarizing the features present in regions of the feature map generated by convolutional layers.

Max Pooling

Max pooling is the most common pooling operation, and it involves sliding a window (usually 2x2 or 3x3 in size) over the feature map and taking the maximum value within the window as the output.

Average Pooling

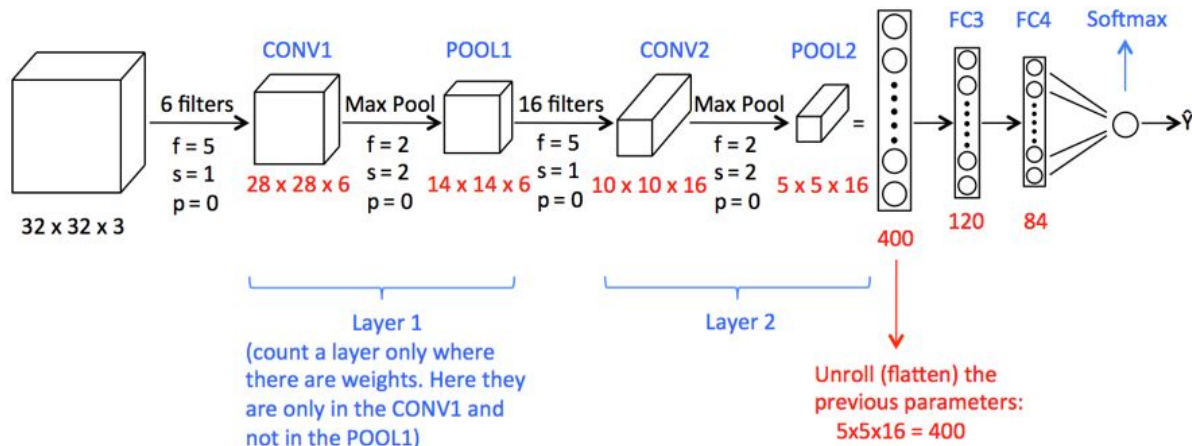
Average pooling takes the average of the elements within the window. It is less common than max pooling



Spatial Calculation

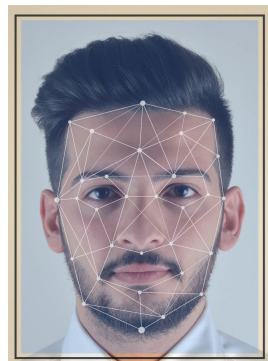
$$n_{out} = \left\lfloor \frac{n_{in} + 2p - k}{s} \right\rfloor + 1$$

n_{in} : number of input features
 n_{out} : number of output features
 k : convolution kernel size
 p : convolution padding size
 s : convolution stride size



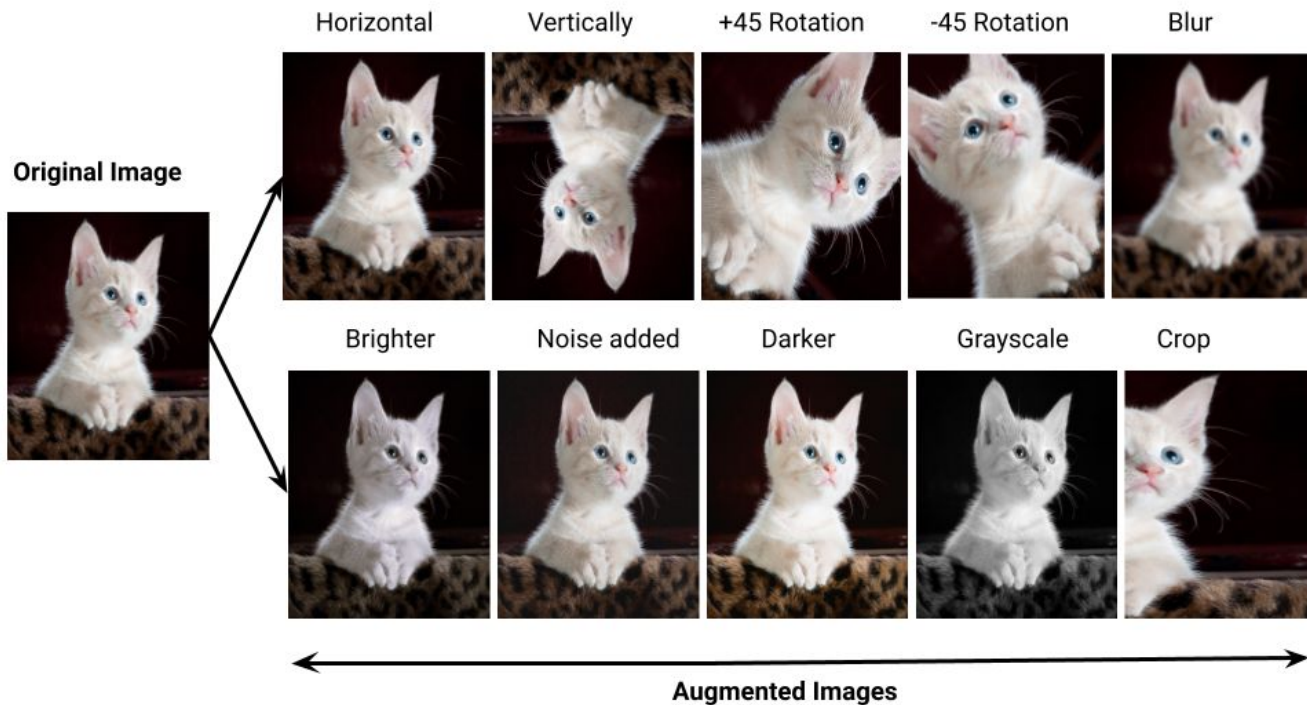
Practical Applications of CNNs

- **Image classification:** Convolutional neural networks are used for image categorization, where images are assigned to predefined categories. One use of such a scenario is automatic photo organization in social media platforms.
- **Object detection:** CNNs are able to identify and locate multiple objects within an image. This capability is crucial in multiple scenarios of shelf scanning in retail to identify out-of-stock items.
- **Facial recognition:** this is also one of the main industries of application of CNNs. For instance, this technology can be embedded into security systems for efficient control of access based on facial features.



Training optimization

Data Augmentation for CV Tasks



Data Augmentation for CV Tasks



Figure 5. Base augmentation applied to training images set

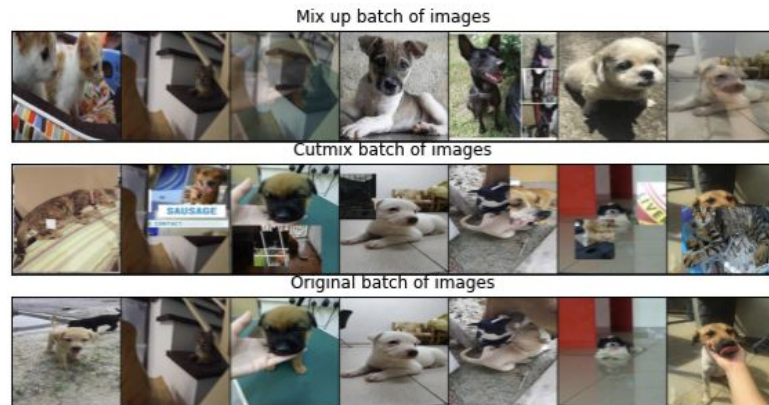


Figure 6. Visualization of first 7 images amongst the batch of image data. We compare original, mixup and cutmix based on our implementation

Batch Normalisation

Normalization is a pre-processing step done before training a machine learning model. Its goal is to make sure that all input features are on a similar scale. For example, one feature might represent height in centimeters (like 170), while another is a ratio (like 0.8).

Why normalize?

- Improve gradient flows (recall: gradients of some non-linear layers are saturated, i.e gradient is small so upstream gradient passed down is small)
- Helps the model learn more effectively
- Cost function will be more symmetrical (links back to gradient descent)

Batch Normalisation

Batch Normalization is a technique used inside a neural network, typically between layers, to normalize the outputs (activations) of those layers during training.

Instead of normalizing the entire dataset, Batch Norm works on mini-batches (small subsets of the data) during training.

Batch Norm standardizes the activations (outputs) of each layer:

1. For each mini-batch, it calculates the mean and standard deviation of the activations.
2. It then normalizes the activations so that they have zero mean and unit variance.

Batch Normalisation

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\dots m}\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

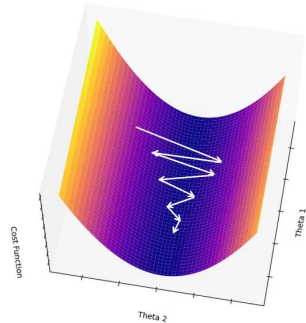
$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

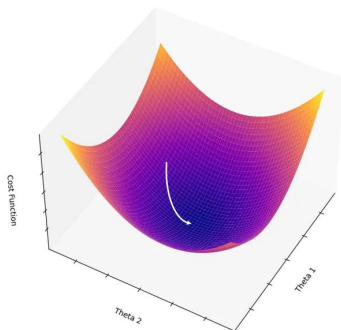
Why Batch Normalisation

Easier to Converge

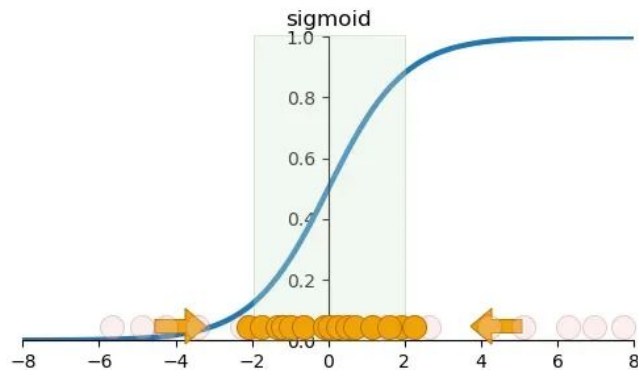
Without Normalization



After Normalization

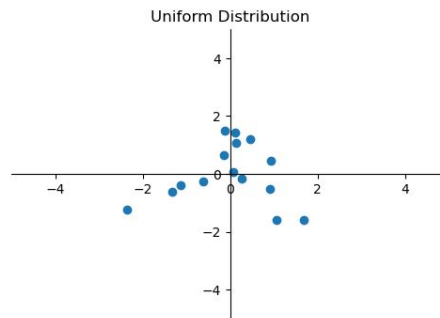


Prevents Vanishing Gradients

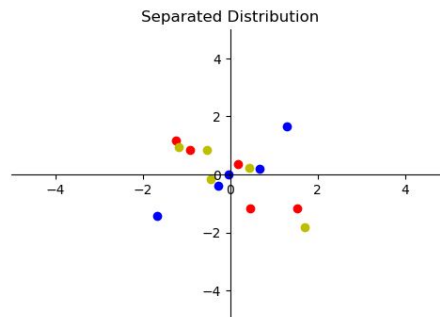


Regularizing effect

Full training set



Batch training set



<https://www.dataleadsfuture.com/visualizing-what-batch-normalization-is-and-its-advantages/>

When to apply BatchNorm?

1. Convolutional Layers — *Very common*

- Placed after a convolution and before the activation:
Conv2D→BatchNorm→ReLU
- Helps stabilize training, reduce internal covariate shift, and allow for higher learning rates.
- Widely used in architectures like ResNet, VGG, etc.

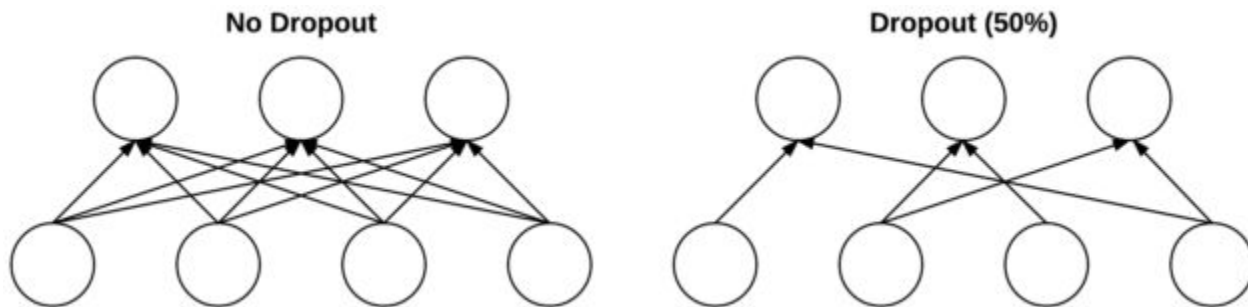
2. Fully Connected / Linear Layers — *Less common, but valid*

- Used in deep networks with multiple dense layers (e.g., MLPs or transformer heads).
- Format:
Linear→BatchNorm1d→ReLU
- Especially helpful when the input features vary significantly in scale.

Regularisation - Dropout

Dropout is a regularization technique used in neural networks to prevent overfitting. It works by randomly "dropping out" (i.e., setting to zero) a fraction of the neurons in a layer during each training step. This forces the network to learn redundant representations and become more robust.

High dropout rates can hinder learning — common values are 0.2–0.5



Code-along

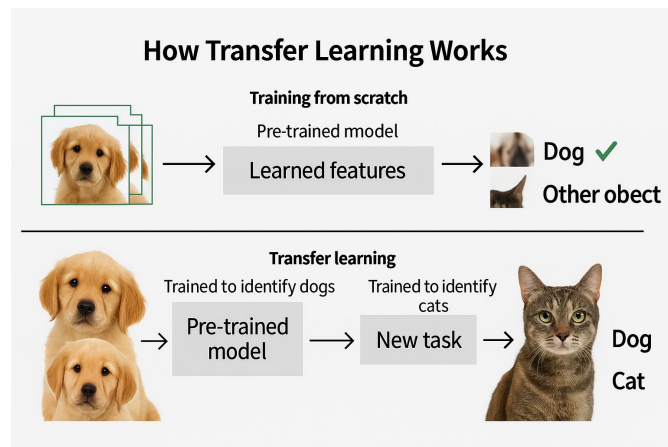


Jupyter Notebook

Part 2: CNNs

Transfer Learning

Transfer learning is a machine learning technique where a model developed for one task is reused as the starting point for a model on a second task. It is a popular approach in deep learning where pre-trained models are used as the starting point on computer vision (and natural language processing) tasks given the vast compute and time resources required to develop neural network models on these problems and from the huge jumps in skill that they provide on related problems.

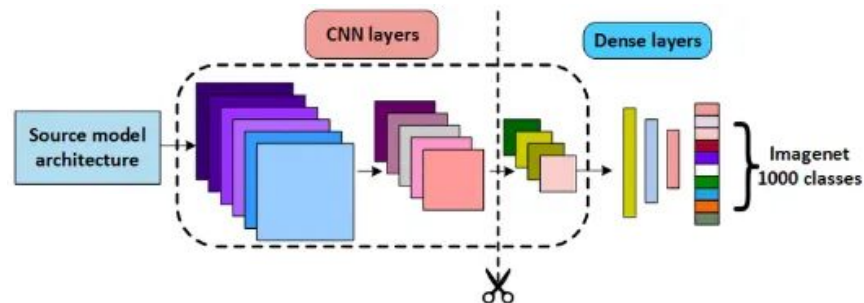


How does Transfer learning work?

The first layers of a CNN learn to recognize general features such as vertical edges, the subsequent layers, the horizontal edges, and then maybe these features are combined to recognize the corners, circles, etc.

These high-level features are independent of the type of entity we need to recognize. Computer vision models don't just "learn" exactly what, for example, cats look like.

Instead, models break down images into small components and learn how those small components combine to make features associated with a particular concept.



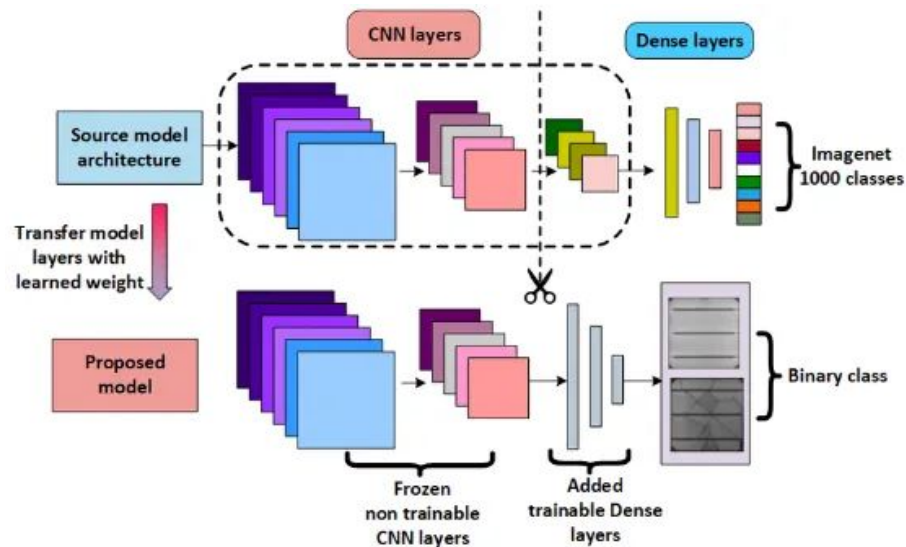
https://www.researchgate.net/figure/Proposed-framework-of-transfer-learning-model_fig3_359273663

How does Transfer learning work?

Recognition in the entity happens at the dense layers that take as input the features extracted from the convolutional layers and learn to categorize in the final class.

To apply transfer learning, we remove the dense layers of the already trained model (since they are layers that have been trained to recognize other classes) and add new ones.

We retrain the new layers in such a way that they specialize in recognizing our classes of interest.



https://www.researchgate.net/figure/Proposed-framework-of-transfer-learning-model fig3_359273663

Benefits of Pre-trained models

Pre-trained models are trained on large benchmark datasets, such as ImageNet, which has over a million images with 1000 categories. Using these models can lead to the following benefits:

1. **Time-saving:** Training a model from scratch requires a significant amount of data and computational resources. Pre-trained models have already learned a lot of features, which can drastically reduce the training time.
2. **Improved performance:** Pre-trained models often increase the performance of neural networks, especially when the amount of training data for the new task is limited.
3. **Feature extraction:** Models pre-trained on large datasets have learned to recognize a variety of features that can be useful for new tasks, even if those tasks involve different classes or categories.

When Transfer Learning may not be ideal

Transfer learning is a powerful technique in deep learning, where you use a pre-trained model on a large dataset and fine-tune it for your specific task. However, there are situations where transfer learning might not be ideal.

Here are some scenarios where transfer learning might not be the best option:

1. Your Task and Dataset Are Very Different from the Pre-trained Model's Task:

- **Problem:** Transfer learning works best when the pre-trained model was trained on a dataset that is similar to your dataset in terms of domain, content, and characteristics. If the pre-trained model was trained on a completely different type of data (e.g., a model trained on natural images like ImageNet being used on medical images), transfer learning may not perform well.
- **Example:** Using a model trained on ImageNet for classifying medical scans (like X-rays or MRI images) might not yield good results because the feature representations learned from natural images may not be applicable to medical images.

When Transfer Learning may not be ideal

2. Insufficient Data for Fine-tuning:

- **Problem:** Transfer learning usually requires some amount of fine-tuning with your specific data. If you have very little data for your task, fine-tuning might lead to overfitting, as the pre-trained model's weights might not generalize well to the small dataset.
- **Example:** If you're trying to fine-tune a model with a few hundred images, the model may not adapt properly and could perform worse than training from scratch.

3. When the Task Requires Domain-Specific Knowledge:

- **Problem:** If your task requires very specialized domain knowledge that the pre-trained model hasn't encountered, transfer learning might not work well. Models trained on general datasets (like ImageNet) might not capture domain-specific features that are crucial for your task.
- **Example:** Using a general pre-trained model for natural language processing tasks on a specific legal document corpus might not work well because the pre-trained model hasn't been exposed to the unique vocabulary and structures of legal texts.

Popular Pre-Trained Models

Common Models and Their Applications

- ResNet (Residual Network): Excellent for image classification tasks.
- EfficientNet: Balances accuracy and efficiency.
- Newer architectures: ViT vision transformers. Uses transformers instead of CNN blocks (talk about this next week)

Code-along



Jupyter Notebook

Part 3: Transfer Learning

End of Lesson - Exit Ticket

Survey at

<https://www.slido.com>

